

Instituto Superior de Engenharia

Politécnico de Coimbra

Sistemas Operativos 2024/2025

P8

Ricardo Duarte Rodrigues, 2022147797, a2022147797@isec.pt

Rodrigo Alexandre Almeida Gonçalves, 2022145597, a2022145597@isec.pt

Índice

Introdução: 3
Programa Manager: 4
Estruturas de dados: 4
Funções principais: 5
Processos e Threads: 8
Comunicação com o feed: 9
Programa Feed: 9
Funções principais: 9
Processos e Comunicação: 10
Makefile: 10
Estrutura do Makefile: 10
Conclusão: 10

Introdução:

O objetivo deste trabalho prático foi implementar uma plataforma de mensagens curtas em C, utilizando os mecanismos do sistema operativo UNIX, com foco na comunicação entre processos. A plataforma foi projetada para permitir a troca de mensagens entre utilizadores, associadas a tópicos, com funcionalidades de subscrição e envio de mensagens persistentes.

O sistema utiliza named pipes para comunicação entre dois programas principais: o manager, que gere os tópicos e os utilizadores, e o feed, que é a interface para os utilizadores interagirem com o sistema.

Programa Manager:

O programa manager é o núcleo do sistema, responsável por gerir tópicos, mensagens e utilizadores. Ele atua como um servidor central que recebe, processa e distribui as mensagens entre os utilizadores subscritos aos tópicos. O manager também gere a criação e remoção de tópicos, além de permitir ao administrador realizar ações como bloquear tópicos e expulsar utilizadores.

Estruturas de dados:

- Message: Armazena informações sobre as mensagens enviadas para os tópicos. Cada mensagem tem:
  - text: o conteúdo da mensagem.
  - author: o nome do utilizador que enviou a mensagem.
  - is_persistent: um indicador que define se a mensagem é persistente (armazenada para os utilizadores que entrarem posteriormente).
  - lifetime: o tempo de vida da mensagem persistente, que decrementa durante a execução do sistema.

```c
typedef struct
{
    char text[MAX_MSG];
    char author[MAX_USERNAME_LEN];
    int is_persistent;
    int lifetime;
} Message;
```

- User: Representa um utilizador do sistema, contendo:
  - username: o nome do utilizador.
  - pid: o identificador do processo do utilizador (usado para enviar sinais ou terminar o processo).
  - client_to_manager_fd e manager_to_client_fd: os descritores de arquivo para os pipes de comunicação entre o utilizador e o manager.
  - next: um ponteiro para o próximo utilizador na lista (não é utilizado de forma eficiente, mas poderia ser útil para uma implementação mais avançada).

```c
typedef struct
{
    char username[MAX_USERNAME_LEN];
    pid_t pid;
    int client_to_manager_fd;
    int manager_to_client_fd;
    struct User *next;
} User;
```

- Topico: Representa um tópico na plataforma. Cada tópico pode ter várias mensagens e subscritores. Cada tópico possui:
  - name: o nome do tópico.
  - messages: um array de mensagens associadas a esse tópico.
  - message_count: o número de mensagens persistentes armazenadas no tópico.
  - is_locked: um flag que indica se o tópico está bloqueado (impede que os utilizadores enviem mensagens para o tópico).
  - subscribers: uma lista de utilizadores subscritos ao tópico.
  - subscribers_count: o número de utilizadores subscritos ao tópico.
  - topics_lock: um mutex para garantir que o acesso ao tópico e seus subscritores seja sincronizado entre múltiplas threads.

```
typedef struct
{
    char name[50];
    Message *messages[100];
    int message_count;
    int is_locked;
    User *subscribers[MAX_USERS];
    int subscribers_count;
    pthread_mutex_t topics_lock;
} Topic;
```

Funções principais:

- add_message_to_topic: Esta função é responsável por adicionar uma nova mensagem a um tópico específico. Ela verifica se o tópico está bloqueado ou se já atingiu o número máximo de mensagens persistentes. Caso contrário, ela aloca memória para a nova mensagem e a adiciona ao tópico. Se a mensagem for persistente, ela será armazenada até o tempo de vida expirar.

```
void add_message_to_topic(Topic *topic, const char *text, const char *author, int is_persistent, int lifetime)
{
    if (topic->message_count >= 5)
    {
        printf("Limite de mensagens persistentes para o tópico '%s' alcançado.\n", topic->name);
        return;
    }
    if (topic->is_locked == 1)
    {
        printf("\nTópico '%s' bloqueado", topic->name);
        return;
    }
    Message *message = malloc(sizeof(Message));
    strncpy(message->text, text, MAX_MSG);
    strncpy(message->author, author, MAX_USERNAME_LEN);
    message->is_persistent = is_persistent;
    message->lifetime = lifetime;

    topic->is_locked = 0;

    if (message->is_persistent)
    topic->messages[topic->message_count++] = message;
}
```

- get_or_create_topic: Verifica se um tópico com o nome fornecido já existe. Se sim, retorna o tópico. Caso contrário, cria um tópico (se o número máximo de tópicos ainda não foi atingido) e retorna o novo tópico.

```
Topic *get_or_create_topic(const char *topic_name)
{
    for (int i = 0; i < topic_count; i++)
    {
        if (strcasecmp(topics[i].name, topic_name) == 0)
        {
            return &topics[i];
        }
    }

    if (topic_count < MAX_TOPICS)
    {
        Topic *new_topic = &topics[topic_count++];
        strncpy(new_topic->name, topic_name, sizeof(new_topic->name));
        new_topic->message_count = 0;
        printf("\nNovo tópico criado: %s", topic_name);
        return new_topic;
    }
    printf("\nNúmero máximo de tópicos alcançado.\n");
    return NULL;
}
```

- notify_subscribers: Quando uma mensagem é enviada para um tópico, esta função é chamada para notificar todos os utilizadores que estão subscritos ao tópico. Ela envia a mensagem para os pipes dos utilizadores subscritos, garantindo que cada um receba a mensagem formatada corretamente.

```
void notify_subscribers(Topic *topic, const char *message, const char *sender)
{
    pthread_mutex_lock(&topic->topics_lock);

    for (int i = 0; i < topic->subscribers_count; i++)
    {
        User *subscriber = topic->subscribers[i];
        if (subscriber)
        {
            char formatted_message[MAX_MSG];
            snprintf(formatted_message, sizeof(formatted_message),
                    "[%s] %s: %s\n", topic->name, sender, message);
            write(subscriber->manager_to_client_fd, formatted_message, strlen(formatted_message) + 1);
            fsync(subscriber->manager_to_client_fd);
        }
    }

    pthread_mutex_unlock(&topic->topics_lock);
}
```

- decrement_lifetime: Esta função executa uma thread separada que decrementa o tempo de vida das mensagens persistentes a cada segundo. Quando o tempo de vida de uma mensagem chega a zero, a mensagem é removida da memória.

```c
void *decrement_lifetime(void *arg)
{
    while (1)
    {

        pthread_mutex_lock(&messages_mutex);

        for (int i = 0; i < topic_count; i++)
        {
            Topic *topic = &topics[i];
            for (int j = 0; j < topic->message_count; j++)
            {
                Message *msg = topic->messages[j];
                if (msg->lifetime > 0)
                {
                    msg->lifetime--;
                }
                if(msg->lifetime <=0){
                    for (int k = j; k < topic->message_count - 1; k++)
                    {
                        topic->messages[k] = topic->messages[k + 1];
                    }
                    topic->message_count--;
                    break;
                }
            }
        }
        pthread_mutex_unlock(&messages_mutex);
        sleep(1);
    }
    return NULL;
}
```

- save_persistent_messages e load_persistent_mensagens: Essas funções são responsáveis por salvar e carregar as mensagens persistentes de um arquivo. Quando o manager é encerrado, ele salva todas as mensagens persistentes restantes no arquivo (especificado pela variável de ambiente FILENAME), e quando é iniciado novamente, ele recarrega as mensagens persistentes que ainda não expiraram.

```c
void save_persistent_messages()
{
    const char *FILENAME = getenv("MSG_FICH");
    FILE *file = fopen(FILENAME, "w");
    if (!file)
    {
        perror("Erro ao abrir arquivo para salvar mensagens persistentes");
        return;
    }
    for (int i = 0; i < topic_count; i++)
    {
        Topic *topic = &topics[i];
        for (int j = 0; j < topic->message_count; j++)
        {
            Message *msg = topic->messages[j];
            if (msg->lifetime > 0)
            {
                fprintf(file, "%s|%s|%s|%d|%d\n",
                        topic->name,
                        msg->author,
                        msg->text,
                        msg->is_persistent,
                        msg->lifetime);
            }
        }
    }

    fclose(file);
    printf("Mensagens persistentes salvas com sucesso.\n");
}
```

- manager_commands_handler: Este é o loop de controlo que permite ao administrador do sistema interagir com o manager. Ele lê os comandos enviados pelo administrador, como listar utilizadores, bloquear tópicos, remover utilizadores ou encerrar o sistema. Esse loop também interage diretamente com os tópicos e utilizadores, executando ações de gestão.

```c
void *manager_commands_handler(void *arg)
{
    char buffer[200];
    while (1)
    {
        printf("\nManager-> ");
        fgets(buffer, sizeof(buffer), stdin);
        buffer[strcspn(buffer, "\n")] = 0;

        char command[50];
        char param[100];

        if (sscanf(buffer, "%s %[^\n]", command, param) == 2)
        {
            if (strcasecmp(command, "remove") == 0)
            {
                remove_user(param);
            }
            else if (strcasecmp(command, "show") == 0)
            {
                // if(get_topic_by_name(param))
                listPersistentMessages(param);
                // Mostrar todas as mensagens persistentes
            }
            else if (strcasecmp(command, "lock") == 0)
            {
                lock_topic(param, 1);
            }
            else if (strcasecmp(command, "unlock") == 0)
            {
                lock_topic(param, 0);
```

Processos e Threads:

O manager executa uma thread dedicada, decrementa_tempo, que continuamente verifica e decrementa o tempo de vida das mensagens persistentes. Isso permite que as mensagens persistentes expirem após o tempo especificado.

Cada utilizador que se conecta ao sistema é gerido por uma thread separada, que escuta por mensagens e executa os comandos enviados pelo utilizador. O uso de mutexes (pthread_mutex_t) é fundamental para garantir que as operações em tópicos e mensagens sejam seguras em um ambiente multithread.

Comunicação com o feed:

A comunicação entre o manager e os programas feed é feita através de named pipes. O manager cria um pipe de comunicação para cada utilizador e espera por comandos ou mensagens. Quando um utilizador envia uma mensagem ou subscreve um tópico, o manager gere o envio dessas mensagens para os subscritores.

Programa Feed:

O feed é a interface do utilizador que permite enviar e receber mensagens, subscrever ou cancelar subscrições de tópicos e interagir com a plataforma. O programa utiliza threads para permitir que o utilizador envie comandos enquanto ainda recebe mensagens de tópicos aos quais está subscrito.

Funções principais:

- listen_for_messages: Esta função executa numa thread separada, lendo continuamente as mensagens do pipe do manager. Quando uma mensagem é recebida, ela é exibida no terminal do utilizador. O uso de threads permite que o utilizador envie comandos enquanto recebe mensagens sem bloquear a execução do programa.

```c
void* listen_for_messages(void* arg) {
    ThreadArgs* args = (ThreadArgs*)arg;
    char message[MAX_MSG_LENGTH];

    while (1) {
        int bytes_read = read(args->manager_to_client_fd, message, sizeof(message));
        if (bytes_read > 0) {
            message[bytes_read] = '\0';
            printf("\n%s\n", message);
        }
    }
    return NULL;
}
```

- sigterm_handler: Trata o sinal SIGTERM enviado pelo manager quando o utilizador é desconectado. Quando o utilizador é removido da plataforma (por exemplo, pelo administrador), o programa termina de forma limpa e avisa o utilizador que foi desconectado.

```c
void sigterm_handler(int signum) {
    printf("\nDesconectado pelo manager.\n");
    exit(0);
}
```

Processos e Comunicação:

O programa feed cria dois pipes dedicados para cada utilizador, um para receber mensagens do manager e outro para enviar comandos para o manager. A comunicação é feita de forma assíncrona, permitindo que o utilizador envie comandos enquanto escuta mensagens.

Makefile:

O Makefile define os processos de compilação dos programas manager e feed. Ele usa o compilador GCC e as flags de compilação -Wall (para exibir todos os avisos) e -g (para gerar informações de debug). Ele também define um alvo clean, que remove os arquivos temporários e os executáveis gerados durante a compilação.

Estrutura do Makefile:

O Makefile compila o manager e o feed usando os arquivos de código-fonte correspondentes. Ele também define os ficheiros a limpar os arquivos compilados e os pipes temporários gerados pela execução do sistema.

Conclusão:

Este trabalho foi uma excelente oportunidade para aplicar conhecimentos de programação em C e conceitos de sistemas operativos, especialmente no que diz respeito à comunicação entre processos e sincronização. A principal dificuldade foi o envio da mensagem de um utilizador para os restantes subscritos a esse tópico.